TEMA 1
KOTLIN DESDE CERO

Programación Multimedia y Dispositivos Móviles
Versión ALUMNADO - prácticas propuestas

Objetivo del tema: aprender los fundamentos de Kotlin entendiendo desde el primer momento para qué pueden servir dentro de una futura aplicación Android. Cada apartado incluye explicación, ejemplo Kotlin, relación con una app y comparación con Java.

Índice

1.1 Introducción a los fundamentos de Kotlin

1.2 Función principal

1.3 Variables

1.4 val vs var: cuándo usarlas

1.5 Tipo Boolean

1.6 Tipos numéricos: Int y Long

1.7 Tipos decimales: Float y Double

1.8 Tipos de texto: Char y String

1.9 Unión de textos: concatenación y String Templates

1.10 Nulabilidad en Kotlin

1.11 Operador Elvis

1.12 Operadores matemáticos

1.13 Operadores de incremento

1.14 Operadores de igualdad

1.15 Operadores lógicos

1.16 Operadores comparativos

1.17 Sentencia if

1.18 Sentencia if else

1.19 Sentencia when

1.20 Ciclo for

1.21 Ciclo while y do while

1.22 Break, continue y return

1.23 Excepciones con throw

1.24 Control de errores con try, catch y finally

1.25 Listas de solo lectura

1.26 Listas mutables

1.27 Map: diccionarios

1.28 Transformaciones: map, zip y flatten

1.29 Filtrar estructuras de datos

1.30 Ordenamiento de listas

1.31 Variables globales vs locales

1.32 Constantes

1.33 Asignación tardía: lateinit

1.34 Inicialización diferida: by lazy

1.35 Firma tus clases y archivos en Android Studio

Prácticas finales integradoras (3)

1.1 Introducción a los fundamentos de Kotlin

Kotlin es el lenguaje principal que utilizaremos para programar la lógica de nuestras aplicaciones Android. Es un lenguaje moderno, conciso y seguro. En este tema todavía no construiremos toda la interfaz: aprenderemos las piezas que después decidirán qué muestra la app, qué datos guarda y cómo responde a las acciones del usuario.

Ejemplo en Kotlin

```
fun main() {
    println("DAM Tasks iniciada")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? En una app, esta lógica terminará dentro de clases, ViewModel o funciones que controlan la interfaz.

Equivalente aproximado en Java

```
public class Main {
    public static void main(String[] args) {
        System.out.println("DAM Tasks iniciada");
    }
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.2 Función principal

La función main() es el punto de entrada de un programa Kotlin convencional. Android no inicia una app llamando directamente a nuestro main(), pero practicar con main nos permite probar la lógica de Kotlin de forma sencilla antes de conectarla a pantallas Android.

Ejemplo en Kotlin

```
fun main() {
    val usuario = "Ana"
    println("Bienvenida, $usuario")
```

```
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Sirve para probar rápidamente algoritmos que luego reutilizaremos en una Activity, ViewModel o composable.

Equivalente aproximado en Java

```
public static void main(String[] args) {
   String usuario = "Ana";
   System.out.println("Bienvenida, " + usuario);
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.3 Variables

Una variable permite guardar un dato con un nombre para utilizarlo después. Una aplicación necesita variables para representar el usuario, una puntuación, el número de mensajes, el precio de un producto o cualquier estado que pueda necesitar.

Ejemplo en Kotlin

```
val nombre = "Mario"
var notificaciones = 2
notificaciones = 3
println("$nombre tiene $notificaciones notificaciones")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Podrían alimentar el nombre mostrado en un perfil y el contador de notificaciones de la app.

Equivalente aproximado en Java

```
String nombre = "Mario";
int notificaciones = 2;
notificaciones = 3;
```

```
System.out.println(nombre + " tiene " + notificaciones + " notificaciones");
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.4 val vs var: cuándo usarlas

val crea una referencia que no puede reasignarse. var permite cambiar el valor. Como regla inicial, usa val siempre que puedas y var solamente cuando el dato deba cambiar. Esto hace el código más predecible.

Ejemplo en Kotlin

```
val idUsuario = 125
var modoOscuro = false
modoOscuro = true
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? El id del usuario puede permanecer fijo mientras que una preferencia como modoOscuro puede cambiar.

Equivalente aproximado en Java

```
final int idUsuario = 125;
boolean modoOscuro = false;
modoOscuro = true;
```

Diferencia clave. val expresa inmutabilidad de la referencia de forma directa; su equivalente conceptual habitual es una referencia final en Java.

1.5 Tipo Boolean

Boolean representa únicamente true o false. Es ideal para estados binarios: sesión iniciada, usuario premium, botón activado, tarea completada, permiso concedido, etc.

Ejemplo en Kotlin

```
val sesionIniciada = true
val esPremium = false

if (sesionIniciada) {
    println("Mostrar pantalla principal")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una app puede decidir qué pantalla mostrar según si la sesión está iniciada.

Equivalente aproximado en Java

```
boolean sesionIniciada = true;
boolean esPremium = false;
if (sesionIniciada) {
   System.out.println("Mostrar pantalla principal");
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.6 Tipos numéricos: Int y Long

Int almacena enteros habituales. Long permite números enteros mucho mayores. Kotlin diferencia los tipos y no convierte automáticamente un Int en Long en todas las situaciones.

Ejemplo en Kotlin

```
val unidades: Int = 12
val idPedido: Long = 4_500_000_001L
println("Pedido $idPedido: $unidades unidades")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Int puede representar cantidades; Long es habitual para identificadores o marcas temporales grandes.

Equivalente aproximado en Java

```
int unidades = 12;
long idPedido = 4_500_000_001L;
System.out.println("Pedido " + idPedido + ": " + unidades + " unidades");
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.7 Tipos decimales: Float y Double

Float y Double representan números con decimales. Double ofrece más precisión y suele ser la opción habitual para cálculos generales. Float necesita el sufijo f. Para dinero real, más adelante veremos alternativas más apropiadas que evitan ciertos errores de precisión binaria.

Ejemplo en Kotlin

```
val valoracion: Float = 4.5f
val distancia: Double = 12.75
println("Valoración: $valoracion - Distancia: $distancia km")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una app puede almacenar una valoración media o una distancia calculada.

Equivalente aproximado en Java

```
float valoracion = 4.5f;
double distancia = 12.75;
System.out.println("Valoración: " + valoracion + " - Distancia: " + distancia + " km");
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.8 Tipos de texto: Char y String

Char representa un solo carácter y se escribe entre comillas simples. String representa una cadena de caracteres y se escribe entre comillas dobles. Los textos de una interfaz suelen manejarse como String.

Ejemplo en Kotlin

```
val inicial: Char = 'A'
val nombre: String = "Android"
println("$inicial - $nombre")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? String se utilizará constantemente para títulos, mensajes, nombres y datos introducidos por el usuario.

Equivalente aproximado en Java

```
char inicial = 'A';
String nombre = "Android";
System.out.println(inicial + " - " + nombre);
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.9 Unión de textos: concatenación y String Templates

Podemos unir textos con +, pero Kotlin ofrece String Templates, que permiten insertar variables con $variable y expresiones con ${expresion}. Normalmente resultan más legibles.

Ejemplo en Kotlin

```
val producto = "Auriculares"
val precio = 39.99
val unidades = 2
val resumen = "$unidades x $producto = ${precio * unidades} €"
println(resumen)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Podemos construir el texto que mostrará una tarjeta de producto o el resumen de una compra.

Equivalente aproximado en Java

```
String producto = "Auriculares";
double precio = 39.99;
int unidades = 2;
```

```
String resumen = unidades + " x " + producto + " = " + (precio * unidades) + " €";
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.10 Nulabilidad en Kotlin

Por defecto, una variable Kotlin no puede contener null. Si un dato puede faltar debemos indicarlo con ?. Esta decisión obliga al programador a tratar explícitamente la ausencia del dato y reduce errores NullPointerException.

Ejemplo en Kotlin

```
val telefono: String? = null
val longitud = telefono?.length
println(longitud)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una API puede devolver un teléfono o foto de perfil ausente. La app debe seguir funcionando.

Equivalente aproximado en Java

```
String telefono = null;
Integer longitud = telefono != null ? telefono.length() : null;
```

Diferencia clave. Kotlin incorpora la nulabilidad al sistema de tipos y obliga a tratarla explícitamente; en Java una referencia puede ser null sin que el tipo lo indique.

1.11 Operador Elvis

El operador ?: proporciona un valor alternativo cuando la expresión de la izquierda es null. Es una forma muy útil de decidir qué mostrar cuando un dato opcional no existe.

Ejemplo en Kotlin

```
val apodo: String? = null
val textoVisible = apodo ?: "Sin apodo"
println(textoVisible)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una pantalla de perfil puede mostrar un texto por defecto si el usuario no ha rellenado un campo.

Equivalente aproximado en Java

```
String apodo = null;
String textoVisible = apodo != null ? apodo : "Sin apodo";
```

Diferencia clave. Kotlin incorpora la nulabilidad al sistema de tipos y obliga a tratarla explícitamente; en Java una referencia puede ser null sin que el tipo lo indique.

1.12 Operadores matemáticos

Los operadores +, -, *, / y % permiten realizar cálculos. % devuelve el resto de una división. Debemos prestar atención al tipo de los operandos: dividir dos Int produce un resultado entero.

Ejemplo en Kotlin

```
val precio = 25.0
val cantidad = 3
val subtotal = precio * cantidad
val envio = 4.99
val total = subtotal + envio
println("Total: $total €")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Son la base de carritos, estadísticas, puntuaciones y cálculos de una app.

Equivalente aproximado en Java

```
double precio = 25.0;
int cantidad = 3;
double subtotal = precio * cantidad;
double envio = 4.99;
double total = subtotal + envio;
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.13 Operadores de incremento

++ aumenta una unidad y -- disminuye una unidad. También existen += y -= para modificar un valor por una cantidad concreta. La diferencia entre contador++ y ++contador importa cuando el valor se utiliza dentro de otra expresión.

Ejemplo en Kotlin

```
var carrito = 1
carrito++
carrito += 2
println("Productos: $carrito")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Permite actualizar contadores de productos, intentos, mensajes o puntos.

Equivalente aproximado en Java

```
int carrito = 1;
carrito++;
carrito += 2;
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.14 Operadores de igualdad

== comprueba igualdad de contenido en Kotlin y != comprueba que dos valores sean diferentes. Kotlin hace que == sea la comparación habitual de valores; internamente gestiona la igualdad de forma segura incluso con null. === comprueba identidad de referencia y se usa mucho menos.

Ejemplo en Kotlin

```
val emailGuardado = "alumno@dam.es"
val emailEscrito = "alumno@dam.es"

if (emailGuardado == emailEscrito) {
    println("El correo coincide")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Es esencial para validaciones, búsquedas y comprobaciones de estado.

Equivalente aproximado en Java

```
String emailGuardado = "alumno@dam.es";
String emailEscrito = "alumno@dam.es";
if (emailGuardado.equals(emailEscrito)) {
    System.out.println("El correo coincide");
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.15 Operadores lógicos

&& significa Y, || significa O y ! niega un Boolean. Permiten combinar varias condiciones para tomar una decisión.

Ejemplo en Kotlin

```
val emailCorrecto = true
val passwordCorrecta = true
val bloqueado = false

val puedeEntrar = emailCorrecto && passwordCorrecta && !bloqueado
println(puedeEntrar)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una pantalla de login puede exigir varias condiciones simultáneamente.

Equivalente aproximado en Java

```
boolean puedeEntrar = emailCorrecto && passwordCorrecta && !bloqueado;
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.16 Operadores comparativos

<, >, <= y >= comparan valores. El resultado siempre es Boolean. Se utilizan para límites de edad, stock, puntuaciones, tamaños y validaciones.

Ejemplo en Kotlin

```
val stock = 4
val cantidadSolicitada = 6

if (cantidadSolicitada > stock) {
   println("No hay suficiente stock")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una tienda puede impedir que el usuario añada más unidades de las disponibles.

Equivalente aproximado en Java

```
if (cantidadSolicitada > stock) {
   System.out.println("No hay suficiente stock");
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.17 Sentencia if

if ejecuta un bloque cuando una condición es true. En Kotlin también puede devolver un valor, por lo que puede sustituir en muchos casos al operador ternario de Java.

Ejemplo en Kotlin

```
val bateria = 18
if (bateria < 20) {
   println("Mostrar aviso de batería baja")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? La app puede mostrar u ocultar avisos según el estado de los datos.

Equivalente aproximado en Java

```
if (bateria < 20) {
   System.out.println("Mostrar aviso de batería baja");
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.18 Sentencia if else

if/else permite elegir entre dos caminos. Podemos encadenar else if cuando existen más situaciones. En Kotlin la estructura puede asignar directamente el resultado a una variable.

Ejemplo en Kotlin

```
val edad = 17
val mensaje = if (edad >= 18) {
   "Registro permitido"
} else {
   "Debes ser mayor de edad"
}
println(mensaje)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Puede generar el mensaje de validación que luego verá el usuario.

Equivalente aproximado en Java

```
String mensaje;
if (edad >= 18) {
   mensaje = "Registro permitido";
} else {
   mensaje = "Debes ser mayor de edad";
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.19 Sentencia when

when permite elegir entre múltiples casos y es más flexible que el switch tradicional de Java. Puede comparar valores, rangos y condiciones, y también devolver un resultado.

Ejemplo en Kotlin

```
val estadoPedido = 2
val texto = when (estadoPedido) {
   0 -> "Recibido"
   1 -> "Preparando"
   2 -> "Enviado"
   3 -> "Entregado"
   else -> "Desconocido"
}
println(texto)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Es muy útil para convertir estados internos en textos, iconos o comportamientos de interfaz.

Equivalente aproximado en Java

```
String texto;
switch (estadoPedido) {
   case 0: texto = "Recibido"; break;
   case 1: texto = "Preparando"; break;
   case 2: texto = "Enviado"; break;
   case 3: texto = "Entregado"; break;
   default: texto = "Desconocido";
}
```

Diferencia clave. Kotlin permite que when devuelva directamente un valor y admite rangos; el switch clásico de Java suele necesitar más estructura.

1.20 Ciclo for

for repite un bloque recorriendo rangos, listas u otras colecciones. En Kotlin no necesitamos el for clásico de Java para la mayoría de recorridos.

Ejemplo en Kotlin

```
val tareas = listOf("Kotlin", "XML", "Emulador")
for ((indice, tarea) in tareas.withIndex()) {
    println("${indice + 1}. $tarea")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Más adelante una colección como esta se mostrará en una lista visual.

Equivalente aproximado en Java

```
for (int i = 0; i < tareas.size(); i++) {
    System.out.println((i + 1) + ". " + tareas.get(i));
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.21 Ciclo while y do while

while repite mientras la condición sea true. do while ejecuta el bloque al menos una vez y comprueba después. Hay que asegurarse de que la condición pueda cambiar para evitar bucles infinitos.

Ejemplo en Kotlin

```
var intentos = 0
while (intentos < 3) {
    println("Intento ${intentos + 1}")
    intentos++
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Puede modelar reintentos o procesos repetitivos, aunque en Android debemos evitar bloquear el hilo de interfaz.

Equivalente aproximado en Java

```
int intentos = 0;
while (intentos < 3) {
   System.out.println("Intento " + (intentos + 1));
   intentos++;
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.22 Break, continue y return

break termina el bucle; continue salta directamente a la siguiente iteración; return termina una función y puede devolver un resultado. Son herramientas distintas y conviene no confundirlas.

Ejemplo en Kotlin

```
fun buscarProducto(productos: List<String>, buscado: String): Boolean {
    for (producto in productos) {
        if (producto.isBlank()) continue
        if (producto == buscado) return true
    }
    return false
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una app puede detener una búsqueda cuando encuentra el elemento solicitado.

Equivalente aproximado en Java

```
static boolean buscarProducto(List<String> productos, String buscado) {
    for (String producto : productos) {
        if (producto.isBlank()) continue;
        if (producto.equals(buscado)) return true;
    }
    return false;
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.23 Excepciones con throw

throw permite lanzar una excepción cuando detectamos una situación que no debe continuar. Debe utilizarse con criterio: para errores excepcionales, no como sustituto de cualquier validación normal.

Ejemplo en Kotlin

```
fun validarEdad(edad: Int) {
    if (edad < 0) {
        throw IllegalArgumentException("La edad no puede ser negativa")
    }
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? La lógica de negocio puede rechazar datos imposibles antes de guardarlos.

Equivalente aproximado en Java

```
static void validarEdad(int edad) {
    if (edad < 0) {
        throw new IllegalArgumentException("La edad no puede ser negativa");
    }
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.24 Control de errores con try, catch y finally

try contiene código que puede fallar, catch responde a una excepción concreta y finally se ejecuta al final haya error o no. Conviene capturar excepciones específicas y no ocultar errores sin analizarlos.

Ejemplo en Kotlin

```
val entrada = "25a"
try {
    val edad = entrada.toInt()
    println("Edad: $edad")
} catch (e: NumberFormatException) {
    println("Introduce un número válido")
} finally {
    println("Validación terminada")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? En una app permite transformar ciertos fallos en mensajes comprensibles sin cerrar la aplicación.

Equivalente aproximado en Java

```
try {
    int edad = Integer.parseInt(entrada);
} catch (NumberFormatException e) {
    System.out.println("Introduce un número válido");
} finally {
    System.out.println("Validación terminada");
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.25 Listas de solo lectura

listOf crea una lista que no ofrece operaciones para añadir o eliminar elementos a través de esa referencia. Es adecuada cuando la colección debe consultarse pero no modificarse.

Ejemplo en Kotlin

```
val categorias = listOf("Móvil", "Tablet", "Portátil")
println(categorias[0])
println("Categorías: ${categorias.size}")
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Puede representar categorías fijas que una pantalla necesita mostrar.

Equivalente aproximado en Java

```
List<String> categorias = List.of("Móvil", "Tablet", "Portátil");
System.out.println(categorias.get(0));
```

Diferencia clave. Kotlin distingue en su API entre interfaces de colección de solo lectura y mutables, lo que hace explícita la intención del código.

1.26 Listas mutables

mutableListOf crea una colección que sí permite añadir, eliminar o reemplazar elementos. Es útil cuando el contenido cambia durante el uso de la app.

Ejemplo en Kotlin

```
val tareas = mutableListOf("Estudiar Kotlin")
tareas.add("Crear pantalla")
tareas.remove("Estudiar Kotlin")
println(tareas)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Es un primer modelo sencillo para una app de tareas antes de utilizar bases de datos y estado observable.

Equivalente aproximado en Java

```
List<String> tareas = new ArrayList<>();
tareas.add("Estudiar Kotlin");
tareas.add("Crear pantalla");
tareas.remove("Estudiar Kotlin");
```

Diferencia clave. Kotlin distingue en su API entre interfaces de colección de solo lectura y mutables, lo que hace explícita la intención del código.

1.27 Map: diccionarios

Map guarda pares clave-valor. Cada clave identifica un valor. mapOf crea un mapa de solo lectura y mutableMapOf uno modificable. Es útil cuando queremos localizar información mediante una clave.

Ejemplo en Kotlin

```
val stock = mutableMapOf(
    "teclado" to 4,
    "raton" to 8
)
stock["monitor"] = 2
println(stock["teclado"])
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Puede relacionar el código de un producto con su stock, o una preferencia con su valor.

Equivalente aproximado en Java

```
Map<String, Integer> stock = new HashMap<>();
stock.put("teclado", 4);
stock.put("raton", 8);
stock.put("monitor", 2);
```

Diferencia clave. Kotlin distingue en su API entre interfaces de colección de solo lectura y mutables, lo que hace explícita la intención del código.

1.28 Transformaciones: map, zip y flatten

map transforma cada elemento y devuelve una nueva colección. zip combina elementos de dos colecciones por posición. flatten aplana una lista de listas. No modifican necesariamente la colección original: producen datos derivados.

Ejemplo en Kotlin

```
val precios = listOf(10.0, 20.0, 30.0)
val conIva = precios.map { it * 1.21 }

val nombres = listOf("Ana", "Luis")
val puntos = listOf(10, 20)
val ranking = nombres.zip(puntos)

val grupos = listOf(listOf("A", "B"), listOf("C"))
val todos = grupos.flatten()
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una app puede convertir modelos a textos visibles, combinar nombres con puntuaciones o reunir resultados de varios grupos.

Equivalente aproximado en Java

```
List<Double> conIva = precios.stream()
   .map(p -> p * 1.21)
   .toList();

List<String> todos = grupos.stream()
   .flatMap(List::stream)
   .toList();
```

Diferencia clave. Kotlin ofrece operaciones de colecciones y lambdas con una sintaxis especialmente compacta; Java moderno dispone de Streams, pero suele requerir más sintaxis.

1.29 Filtrar estructuras de datos

filter conserva únicamente los elementos que cumplen una condición. La expresión entre llaves es una lambda; cuando tiene un único parámetro Kotlin permite llamarlo it.

Ejemplo en Kotlin

```
val precios = listOf(15.0, 120.0, 35.0, 250.0)
val asequibles = precios.filter { it <= 100.0 }
println(asequibles)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una pantalla de productos puede mostrar solo los que cumplen el filtro elegido por el usuario.

Equivalente aproximado en Java

```
List<Double> asequibles = precios.stream()
   .filter(p -> p <= 100.0)
   .toList();
```

Diferencia clave. Kotlin ofrece operaciones de colecciones y lambdas con una sintaxis especialmente compacta; Java moderno dispone de Streams, pero suele requerir más sintaxis.

1.30 Ordenamiento de listas

sorted y sortedDescending ordenan valores comparables. sortedBy y sortedByDescending permiten indicar qué propiedad se utilizará. El resultado es una nueva lista ordenada.

Ejemplo en Kotlin

```
val puntuaciones = listOf(40, 10, 90, 60)
val ranking = puntuaciones.sortedDescending()
println(ranking)
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una app puede ordenar precios, fechas, puntuaciones o nombres según la opción elegida.

Equivalente aproximado en Java

```
List<Integer> ranking = new ArrayList<>(puntuaciones);
ranking.sort(Comparator.reverseOrder());
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.31 Variables globales vs locales

Una variable local vive dentro de una función o bloque y su alcance queda limitado a esa zona. Una propiedad de clase puede ser accesible por distintos métodos del objeto. Evita convertir todo en estado global: dificulta comprender y probar la aplicación.

Ejemplo en Kotlin

```
class Sesion {
    var usuarioActual: String? = null

    fun mostrarSaludo() {
        val saludo = "Hola"
        println("$saludo ${usuarioActual ?: "invitado"}")
    }
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? En Android el estado debe colocarse en el lugar adecuado, por ejemplo una clase o ViewModel, en vez de abusar de variables globales.

Equivalente aproximado en Java

```
class Sesion {
    String usuarioActual = null;

    void mostrarSaludo() {
        String saludo = "Hola";
        System.out.println(saludo + " " +
            (usuarioActual != null ? usuarioActual : "invitado"));
    }
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.32 Constantes

Una constante representa un valor conocido y fijo. En Kotlin, const val se utiliza para constantes en nivel superior o dentro de object/companion object y solo admite determinados tipos básicos y String.

Ejemplo en Kotlin

```
const val MAX_INTENTOS = 3
const val NOMBRE_APP = "DAM Tasks"

fun main() {
    println("$NOMBRE_APP permite $MAX_INTENTOS intentos")
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Puede definir límites o claves fijas que utiliza la aplicación.

Equivalente aproximado en Java

```
static final int MAX_INTENTOS = 3;
static final String NOMBRE_APP = "DAM Tasks";
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.33 Asignación tardía: lateinit

lateinit permite declarar una propiedad no nula que se inicializará más tarde. Solo se aplica a propiedades var de tipos no primitivos compatibles. Si se accede antes de inicializarla se produce UninitializedPropertyAccessException, por lo que no debe utilizarse para evitar pensar en la nulabilidad.

Ejemplo en Kotlin

```
class PantallaPerfil {
    lateinit var nombreUsuario: String

    fun cargar() {
        nombreUsuario = "Lucía"
    }

    fun mostrar() {
        println(nombreUsuario)
    }
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? En Android aparece en ciertos patrones de inicialización, por ejemplo referencias que solo están disponibles después de una fase del ciclo de vida.

Equivalente aproximado en Java

```
class PantallaPerfil {
   String nombreUsuario;

   void cargar() {
      nombreUsuario = "Lucía";
   }
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.34 Inicialización diferida: by lazy

by lazy calcula un valor la primera vez que se utiliza y conserva el resultado para usos posteriores. La propiedad se declara normalmente con val. Es útil cuando crear el objeto cuesta trabajo y quizá no sea necesario.

Ejemplo en Kotlin

```
val configuracion by lazy {
    println("Cargando configuración...")
    "Configuración preparada"
}

fun main() {
    println("App iniciada")
    println(configuracion)
    println(configuracion)
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Una app puede retrasar la creación de un recurso hasta el momento en que realmente se necesita.

Equivalente aproximado en Java

```
// Java suele resolverlo manualmente:
private String configuracion;

String getConfiguracion() {
    if (configuracion == null) {
        configuracion = "Configuración preparada";
    }
    return configuracion;
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

1.35 Firma tus clases y archivos en Android Studio

Android Studio permite añadir encabezados de archivo y comentarios de documentación. En un proyecto educativo es útil identificar autor, propósito y responsabilidad de una clase. Esto no es la firma criptográfica del APK: es documentación del código. Para documentación reutilizable, Kotlin utiliza KDoc con /** ... */.

Ejemplo en Kotlin

```
/**
 * Gestiona operaciones básicas del inventario.
 * Autor: Alumno/a DAM
 */
class InventarioManager {
    fun totalProductos(): Int = 0
}
```

Lectura del ejemplo. Lee el código de arriba hacia abajo: las declaraciones crean o reciben datos; las expresiones los transforman o comprueban; y println nos permite verificar el resultado antes de conectar esa lógica a una interfaz Android.

¿Para qué serviría en una app? Ayuda a mantener un proyecto Android legible cuando empieza a crecer y participan varias personas.

Equivalente aproximado en Java

```
/**
 * Gestiona operaciones básicas del inventario.
 * Autor: Alumno/a DAM
 */
public class InventarioManager {
    public int totalProductos() {
        return 0;
    }
}
```

Diferencia clave. La idea es equivalente en ambos lenguajes, pero Kotlin suele reducir código repetitivo y expresar la intención con menos sintaxis.

Prácticas finales integradoras

Estas tres prácticas reúnen los contenidos del tema. Están diseñadas como lógica de funcionalidades que en temas posteriores conectaremos a pantallas Android.

Práctica final 1 - Registro y acceso de usuarios

Construye la lógica previa de una futura app de registro/login. Debe guardar nombre, email, edad, contraseña y si el usuario está bloqueado. Valida campos vacíos, edad mínima y longitud de contraseña; usa operadores lógicos y comparativos; genera el mensaje final con if/else o when; trata un apodo opcional con nulabilidad y Elvis; y controla una edad introducida como texto mediante try/catch. La salida por consola debe representar los mensajes que posteriormente vería el usuario en la interfaz.

Entrega: código Kotlin ejecutable, salida de prueba y breve explicación de cómo conectarías esta lógica a una futura interfaz Android.

Práctica final 2 - Inventario de una tienda

Crea la lógica de una futura app de inventario. Utiliza una MutableList para productos, un Map para relacionar nombres y stock, operaciones para añadir/eliminar, filter para obtener productos con stock, sorted para ordenar, map para crear textos visibles y un bucle para mostrar el resumen. Añade una constante de stock mínimo y genera un aviso cuando un producto esté por debajo de ella.

Entrega: código Kotlin ejecutable, salida de prueba y breve explicación de cómo conectarías esta lógica a una futura interfaz Android.

Práctica final 3 - DAM Tasks: motor de una futura app Android

Desarrolla el motor de una futura app de tareas. Debe permitir añadir tareas rechazando texto vacío, eliminarlas, contar tareas, filtrar por una palabra, ordenar alfabéticamente y generar un mensaje de estado con when. Incluye una propiedad opcional para el nombre del usuario, usa Elvis para mostrar Invitado cuando falte y organiza la lógica en funciones. Como ampliación, identifica qué datos serían variables de pantalla y cuáles podrían ser constantes.

Entrega: código Kotlin ejecutable, salida de prueba y breve explicación de cómo conectarías esta lógica a una futura interfaz Android.